

Issue Tracking System Project

1 CONTENTS

2	Overview.....	2
2.1	Objectives.....	2
3	Application.....	2
3.1	Minimal Requirements.....	2
3.2	Issue Fields	3
3.3	Architecture Overview	3
3.3.1	MVC Architecture	4
4	System implementation	4
4.1	Text-Based Version	4
4.1.1	Design	4
4.1.2	Testing.....	5
4.1.3	Other Considerations.....	5
4.2	REST Endpoints.....	5
4.2.1	Design	5
4.2.2	Testing.....	6
4.2.3	Other Considerations.....	6
4.3	Web Client.....	6
5	Milestones	7
5.1	Text-based Issue Tracking System	7
5.2	REST Endpoints.....	7
5.2.1	Tips for Milestone	7
5.3	Web Application.....	8
5.3.1	Notes.....	8
6	Submission and Grading.....	9
6.1	Submission	9
6.2	Text-based Issue Tracking System	9
6.3	REST Endpoints.....	10
6.4	Web Application.....	10

2 OVERVIEW

In this project your team will create a web-based issue tracking system.

The project consists of three milestones:

1. Create a text-based version of an issue tracking system with a MVC architecture.
2. Convert the text-based version to a service that uses the REST architecture.
3. Create a web client interface for the service.

2.1 OBJECTIVES

Some of the objectives of this project are:

- Work collaboratively as a team to build a useful real-world product.
- Gain experience in service-oriented software engineering.
- Follow an agile methodology to develop a software product.
- Use good software engineering practices to develop software.

3 APPLICATION

The application is an issue tracking system. Some examples of issue tracking systems include:

- GitLab (<http://gitlab.cs.uleth.ca/>) or GitHub (<http://github.com>)
- Bugzilla (<https://www.bugzilla.org/>)
- Jira (<https://www.atlassian.com/software/jira>)

3.1 MINIMAL REQUIREMENTS

The system will provide (at minimum) the following functionality:

- Issues
 - Create a new issue
 - Get an issue by id
 - List all or a subset of issues
 - Assign an issue to a user
 - Remove a user from an issue
 - Update any field of an issue
 - Delete an issue
- Comments
 - Add a comment to an issue
 - Get a comment from an issue
 - Get all of an issue's comments
 - Update a comment of an issue
 - Delete a comment of an issue
- Users

- Add a user
 - List all users
 - Remove a user
- Search
 - Find an issue

Each team member will add one feature of their own choosing. The team is to coordinate the additional features so that two team members do not choose the same feature. Example features include:

- User profiles (add, list, update, delete)
- Status (add, list, update, delete) Example statuses: New, Assigned, Fixed, WontFix
- Labels (add, list, update, delete)
- Priority (add, list, update, delete)
- Milestones (add, list, update, delete)
- Grouping of users into roles (add, list, update, delete)
- Issue description templates (add, update, delete)

You can get ideas for features by looking at:

- Bugzilla REST API (https://wiki.mozilla.org/Bugzilla:REST_API)
- Jira REST API (<https://docs.atlassian.com/software/jira/docs/api/REST/latest/>)

The following items are not recommended as they could become a functionality quagmire from which your team may sink and never be seen again, beyond the scope of the project, or not worth the effort for what benefit they will provide.

- User authentication (i.e. user names and passwords)
- Attachments
- Use of encryption (military-grade or otherwise) of data

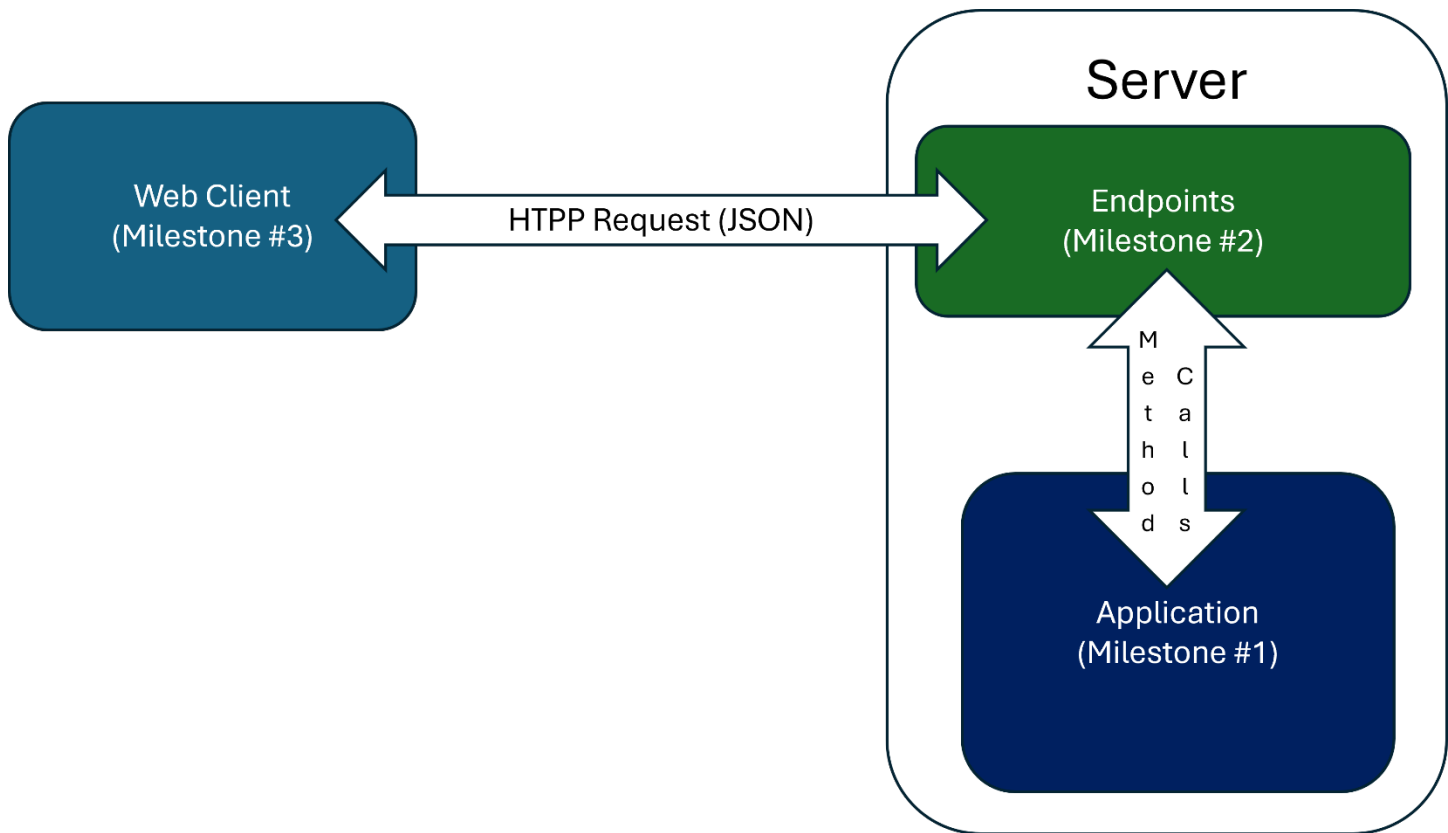
3.2 ISSUE FIELDS

An issue is to have the following fields (at a minimum):

- Identification number
- Title
- Description (this will be Comment #1)
- AssignedTo (a user id)
- Comments

3.3 ARCHITECTURE OVERVIEW

The following diagram shows an overview of the components of the project and how they interact.



3.3.1 MVC Architecture

The application uses the MVC architecture in the following way:

- Model: The implementation that you created in Milestone #1.
- Controller: The implementation of the endpoints in Milestone #2.
- View: The web client (i.e. web pages) in Milestone #3.

4 SYSTEM IMPLEMENTATION

4.1 TEXT-BASED VERSION

Create a text-based issue tracking system. The text interface will prompt the user for an action (e.g. create an issue) and then prompt the user for the needed information (e.g. Issue title, issue description, assigned user).

4.1.1 Design

- The implementation uses the Model-View-Controller architecture.
 - Model classes:
 - Found in a subfolder called "model" (e.g. `src/model/Issue.cpp`)
 - Controller classes:
 - Found in a subfolder called "controller" (e.g. `src/controller/IssueTracker.cpp`)
 - View classes:

- Found in a subfolder called “view” (e.g. `src/view/TextUI.cpp`)
- Dependency Injection is used to tie the Controller to specific Models and Views.
- Use one or more text files for persistence (i.e. storing the issue data)
- The instructor must approve the use of libraries outside of STL.

4.1.2 Testing

- Use unit testing and mocking to test your application.
 - The only classes that should not have > 90% coverage by automated testing are the “View” classes, which contain the `cin` statements.
 - Suggestions for test scaffolding:
 - Use unit tests as drivers for the Controller(s) and Model(s).
 - Use stubs or mocks for the Models to test the Controller(s).
 - Use a dummy file for testing.

4.1.3 Other Considerations

- A template for the project is provided for you as a starting point.
 - Edit the files (e.g. `Makefile`) for your particular application as needed.
 - One team member forks the repo and adds the team members as `Maintainers`.
- Create a `README.md` file in the root of the repository with the following sections and information:
 - Contributions:
 - A subsection for each contributing team member
 - Their assigned responsibilities for the milestone.
 - Links to example issues, merge requests and/or commits showing they completed their responsibilities.
 - Testing: Explain the testing approach used by the team
 - How was the Controller tested (Stubs? Mocks?)
 - A high-level description of what is tested by each test suite. (e.g. “`TestIssue.cpp` tests the functionality related to issues”)
- Individual contributions to the project will be assessed by examining:
 - Discussion with the team about completing the milestone.
 - Completion of issues in GitLab.
 - Commits for issues.
 - Merge requests from developer-created branches into `main`

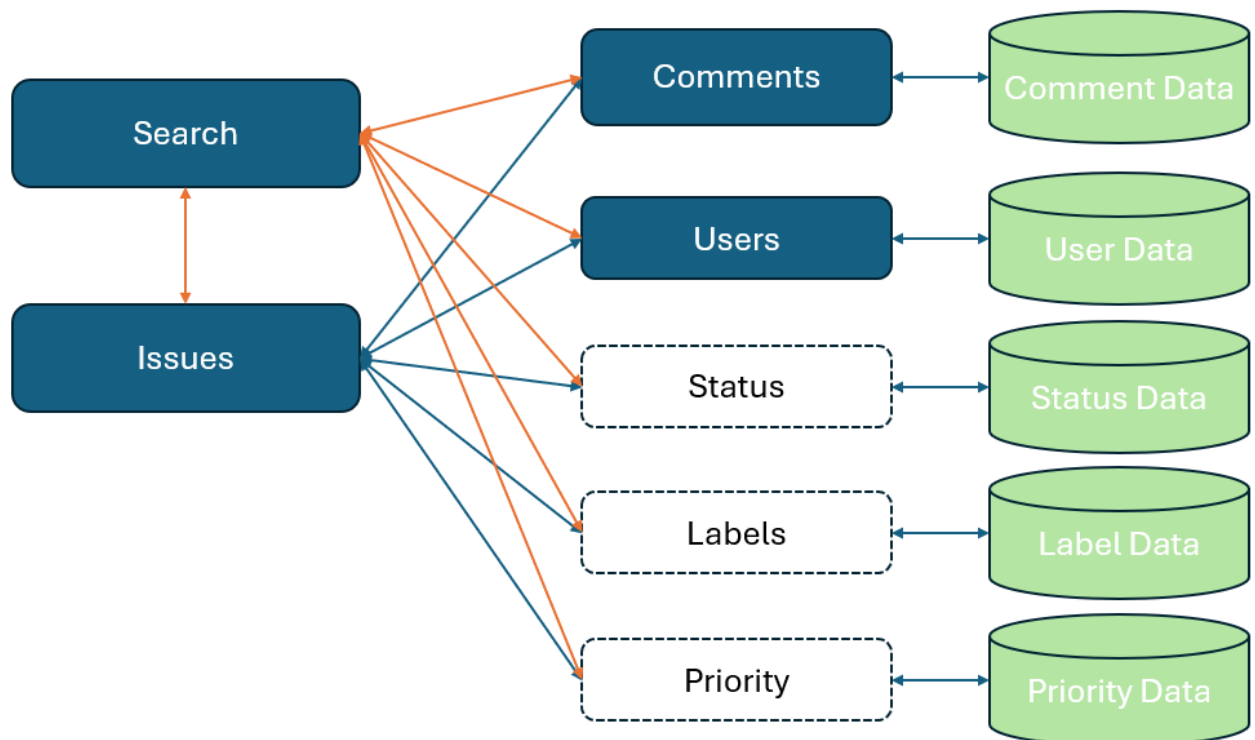
4.2 REST ENDPOINTS

Evolve your text-based issue tracking system into a RESTful service. The message format is JSON.

Each of the items in Section 3.1 is to have its own REST endpoint(s).

4.2.1 Design

- The REST endpoints are expected to call the methods of Model(s).
- The RESTful application is to be created as a set of microservices. An example architecture is shown in the following image.



4.2.2 Testing

- Oat++ provides a unit testing framework for endpoints.
 - It is recommended that you use it, but it is not required.

4.2.3 Other Considerations

- All REST endpoints are to have corresponding *Swagger* UI documentation.
 - This can be used for testing the endpoints.

4.3 WEB CLIENT

The web client can be implemented using whatever technology your team is comfortable with. However, the application must run in the CS labs.

A simple, nice-looking HTML/JavaScript webpage is sufficient to meet the project's needs. If a team wants to go further and use JavaScript frameworks like AngularJS, React or Vue, they may do so. However, the project's primary focus is creating a Client/Server application using REST, so the team's effort is better spent on providing functionality rather than creating a flashy UI.

5 MILESTONES

5.1 TEXT-BASED ISSUE TRACKING SYSTEM

- A working version of your issue tracking system that uses:
 - Model-View-Controller architecture
 - Dependency Injection
- A passing pipeline that does:
 - Code coverage
 - Static Analysis
 - Style Checking
 - Memory-leak testing
 - Documentation generation

5.2 REST ENDPOINTS

- A set of REST endpoints that provide the functionality listed in Section 3.1.
- The handling of malformed requests.
- A passing pipeline that does:
 - Compiles each of the servers
 - Documentation generation

Demonstration of the endpoints will be done using Swagger UI.

5.2.1 Tips for Milestone

- An example RESTful application using microservices is provided.
 - It is suggested that you use this as a starting point for your own application.
 - Simpler example RESTful applications from the Oat++ documentation are:
 - <https://oatpp.io/docs/start/step-by-step/#simplest-project>
 - <https://oatpp.io/docs/start/step-by-step/#well-structured-project>
- You will need to do the following:
 - Create DTOs for data transfer between your services and client.
 - It is likely that you will create a DTO for each of your Models in your application and have a method in your Controller (i.e. REST endpoint implementation) class(es) that translates from the application Model class to a DTO and vice versa.
 - Create endpoints for each “resource” needed for your application.
 - In principle, an endpoint does the following:
 - Translates input information from the GET or POST request.
 - Uses the input information to call one or more Model methods.
 - Translates the return value(s) from the method call(s) into a response DTO.
- Most of your work will be in the following folders:
 - controller (endpoint specification)
 - dtos (data transfer objects used by endpoints)
- Update the following in `server/ServerComponents.hpp`:
 - Port to match that listed in the `Course Project Ports on Moodle`.

- Update the following in `server/Server.cpp`:
 - Controllers to be your Controller class.
 - You can have multiple controllers registered in your application. This can be useful if your team wants to distribute the responsibilities for implementing specific endpoints to different team members who create their controller implementations in separate files in `controller`.
- The classes for your application (i.e. the code called by the endpoints that was created in Milestone #1) is expected to be in the `service` directory.

5.2.1.1 Testing

If you choose to use the Oat++ testing framework:

- See <https://oatpp.io/docs/start/step-by-step/#testing-oat-application> for documentation about how to do testing with the `oatpp-testing` framework.
- `Test.cpp` contains the code that specifies which tests are to be run. These classes will have the `onRun()` method.
- `app/MyApiClient.cpp` defines the calls to the endpoints (i.e. the GET or POST call). These will be used in the implementation of the test cases.
 - `app/TestComponent.hpp` should not need changes.
- A test case is one class (e.g. `CircleAreaTest`) that has a definition of the method `onRun()`.
 - The implementation of the `onRun()` method will call the endpoints defined in `app/MyApiClient.cpp`
 - The assertions (`OPP_ASSERT`) are used to check that the response returned by the endpoint has the data that is expected.

5.3 WEB APPLICATION

Demonstrate:

- A web client that allows a user to perform the actions in Section 3.1, as well as the actions for the additional functionality selected by team members.
- A passing pipeline that does:
 - Code coverage
 - Style Checking
 - Documentation generation

5.3.1 Notes

- The web client does not need to use any JavaScript frameworks (e.g. React, VueJS, AngularJ). A simple HTML and JavaScript page is enough.
 - Look at the `calcREST` project for an example of such a page.
 - https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Client-side_web_APIs may also be helpful.

6 SUBMISSION AND GRADING

6.1 SUBMISSION

- Submit the URL of your GitLab repository to Moodle.
 - This is to avoid confusion about which repository is the team's "official" repository.
- All changes are to be merged into the repository's `main` branch.
 - Only the `main` branch will be used for grading.
- Code coverage, style checking, and documentation generation are expected to be carried over from the previous milestones.
 - This provides an opportunity to improve these aspects from prior milestones if needed.
 - Any changes to the endpoints (e.g. adding a new endpoint) might require updating your unit tests to maintain coverage.
 - Doxygen is used to generate documentation for the Models (and Controller and View for the text-based version).
 - Swagger UI is used to document the endpoints.

6.2 TEXT-BASED ISSUE TRACKING SYSTEM

- A working application [35 marks].
 - Program compiles [5 marks]
 - Manipulate issues
 - Create [2 marks]
 - Modify [2 marks]
 - Delete [2 marks]
 - Manipulate comments
 - Create [2 marks]
 - Modify [2 marks]
 - Delete [2 marks]
 - Search for an issue
 - By id [2 marks]
 - By user [2 marks]
 - Manages invalid input from the user [4 marks]
 - Additional Functionality #1 [5 marks]
 - Additional Functionality #2 [5 marks]
 - Additional Functionality #3 [5 marks]
 - Additional Functionality #4 [5 marks]
- Application uses Model-View-Controller architecture and Dependency Injection. [20 marks]
 - Model class(es) [5 marks]
 - Controller class(es) [5 marks]
 - View class(es) [5 marks]
 - Dependency Injection [5 marks]
- Testing [20 marks]
 - Code coverage >90% for Controller and Model classes [10 marks]

- Documentation in README of how testing was conducted [10 marks]
- Code Quality [15 marks]
 - Static Analysis [5 marks]
 - Style Analysis [5 marks]
 - Memory Leak Checking [5 marks]
- Documentation [10 marks]
 - Class purpose and role in MVC.
 - Method intent.
 - Parameter purpose.
 - Return value description.
- Teamwork [Adjustment to individual grades as needed]
 - Based on the contents of the README.md file and discussion with the team.

6.3 REST ENDPOINTS

- Servers compile (e.g. `make its`) [5 marks]
- Server does not crash during demonstration [15 marks]
- Manipulate issues
 - Create [2 marks]
 - Modify [2 marks]
 - Delete [2 marks]
- Manipulate comments
 - Create [2 marks]
 - Modify [2 marks]
 - Delete [2 marks]
- Search for an issue
 - By id [2 marks]
 - By user [2 marks]
- Additional Functionality #1 [5 marks]
- Additional Functionality #2 [5 marks]
- Additional Functionality #3 [5 marks]
- Additional Functionality #4 [5 marks]
- Handle a malformed request gracefully (i.e. REST endpoint returns a meaningful message) [4 marks]

6.4 WEB APPLICATION

- Servers compile (e.g. `make its`) [5]
- Manipulate issues
 - Create [2 marks]
 - Modify [2 marks]
 - Delete [2 marks]
- Manipulate comments
 - Create [2 marks]
 - Modify [2 marks]
 - Delete [2 marks]

- Search for an issue
 - By id [2 marks]
 - By user [2 marks]
- Additional Functionality #1 [5 marks]
- Additional Functionality #2 [5 marks]
- Additional Functionality #3 [5 marks]
- Additional Functionality #4 [5 marks]